

AI SYSTEMS ENGINEERING — PORTFOLIO PROJECT

Augur v2

OSS dependency risk intelligence — ranking PyPI and npm packages by near-term vulnerability risk, and proving whether it beats a naive baseline.

PREPARED FOR	PROFILE	DURATION	BUILD WINDOW
Alae Rhazouani	AI Systems Engineer	12 weeks, solo build	3 months, 2026

What Augur does and why it matters

Augur monitors the top 10,000 packages on PyPI and npm — contributor activity, release patterns, dependency graphs, OpenSSF Scorecard checks and advisory history — and ranks them by the risk of a vulnerability advisory landing in the next 90 days. A separate module watches new releases for maintainer-takeover patterns. A tool-using assistant answers questions about a dependency tree with every claim tied to a retrieved advisory ID. The whole system re-scores itself weekly against newly published advisories, and CI blocks changes that make it worse.

10K

PACKAGE UNIVERSE

90DAY PREDICTION
WINDOW**3**

BASELINES TO BEAT

12WEEKS, SOLO
EXECUTION

The problem with existing tools

Dependabot, Snyk and Renovate are reactive — they match your dependencies against advisories that are already public. Augur reframes the question as triage: given finite review time, which dependencies deserve attention first, before anything is disclosed?

Augur's approach

Rank packages by near-term risk from public repository-health signals, and measure that ranking against a popularity-only baseline at every step. Ranking problems can be evaluated honestly; "we predict CVEs" cannot.

The falsifiable thesis

The research question is stated so that it can fail: **do public repository-health signals rank packages by near-term vulnerability risk better than popularity alone?** Every model is measured against a popularity-only baseline, and the lift over that baseline is the headline number rather than a raw precision figure.

Three outcomes are all presentable. Clear lift means a working predictor with defensible numbers. Small lift means a feature-level explanation of what carries signal and what is noise, supported by SHAP attribution. No lift is a rigorous negative result — that public health signals add nothing beyond popularity, and why — which interviews better than an inflated positive one. Alongside it stands the engineering thesis: an AI system is production-ready only when it is observable, continuously evaluated and gated in CI.

The two traps this design exists to avoid

Confounding. Popular, actively maintained packages receive more advisories because more people audit them. A naive model learns "busy and popular equals risky", which is the opposite of the intended story. Hence the mandatory popularity baseline at every rung of the model ladder.

Leakage. Fix commits and security releases usually land before an advisory is published, so a model anchored on publication date quietly learns to detect the fix already in progress. Augur anchors labels on the earliest disclosure signal and drops a seven-day blackout window before it. Both traps are tested with explicit ablations, not assumed away.

Two tracks, not one problem

Version 1 of this plan mixed two different problems under one model. Most advisories describe ordinary bugs found by researchers and fuzzers, which correlate with popularity, code size and scrutiny. Maintainer takeovers are rare, behaviourally distinct, and too few to train on. They are now separated, each with a matching label, method and metric.

	TRACK A — RISK RANKING	TRACK B — TAKEOVER DETECTION
Question	Which packages get an advisory in the next 90 days?	Does this release look like a hijack or sabotage?

	TRACK A — RISK RANKING	TRACK B — TAKEOVER DETECTION
Typical cause	Ordinary bugs found by researchers and fuzzers	Hijacked accounts, malicious maintainers, protestware
Label source	OSV advisories for PyPI and npm — thousands of positives	8–12 hand-curated incidents — too few to train on
Method	Supervised ML: LightGBM against three baselines	Transparent rule-based detectors with tuned thresholds
Metric	PR-AUC, precision@k, lift over popularity baseline	Detection lead time, false alarms per 1,000 package-months
Main risk	Leakage and popularity confounding	Overfitting rules to already-known stories

Free data sources — bulk for history, APIs for the daily refresh

SOURCE	WHAT IT PROVIDES	ACCESS
OSV.dev bulk export	Ecosystem-native advisories for PyPI and npm with published dates and fixed versions — the label source	Free zip download per ecosystem
GH Archive	Historical commits, contributors, issues and releases per repo, queryable by date partition	Free via BigQuery public dataset
deps.dev (Google)	Package-to-repo mapping, versions and full dependency graph for PyPI and npm	Free API and BigQuery dataset
OpenSSF Scorecard	Weekly security-practice scores: branch protection, code review, fuzzing, pinned dependencies	Free via BigQuery public dataset
PyPI + npm registries	Download trends, release history, per-version publisher identity, maintainer changes	Free, no auth for read access

BigQuery's free tier covers 1 TB of queries per month, which is ample if every query filters by date partition and repo list and materializes small intermediate tables. The live GitHub API is used only for the daily refresh, never for historical reconstruction.

The honest demo: event-stream, not XZ Utils

XZ Utils is the most studied supply chain attack in history, but it is a C library — not a PyPI or npm package — so it sits outside the scope of this system. More importantly, a detector designed after knowing the XZ story and then backtested on XZ is hindsight, not evidence, and a security interviewer will say exactly that.

The replacement demo is **event-stream** (npm, 2018): a real maintainer handover followed by a new dependency carrying a malicious payload, weeks before public discovery. The detectors are tuned on three design incidents, frozen, and then run once against a held-out set of npm and PyPI incidents with a 500-package control set for false alarms. XZ is credited only as the inspiration for the maintainer-trust features.

Five components, one custom orchestrator

Each component is named for what it actually is. Version 1 called five things "agents", but two of them were an ETL job and an evaluation pipeline, and senior engineers discount projects that label cron jobs as agents. Only the assistant is an agent. The custom Python orchestrator stays — deliberately not LangChain or LangGraph — because shared state, intent routing, retries with backoff and a graceful fallback chain are where the real coordination work is.

COMPONENT 01 · INGESTION PIPELINE

Historical backfill and daily refresh

Backfill from BigQuery: GH Archive activity, Scorecard checks and download history, joined to the 10,000-package universe through deps.dev. Daily refresh from the registries and the GitHub API. Every record is validated against a Pydantic schema before it is written to TimescaleDB, whose hypertables suit the one-row-per-package-per-week structure. Cron first; Airflow later only if the schedule allows.

COMPONENT 02 · RISK MODEL SERVICE

Weekly ranking — Track A

Trains and serves the risk ranking over seven feature families. LightGBM is benchmarked against the three baselines, calibrated with isotonic regression, and every run is logged to MLflow. Promotion from Staging to Production requires beating the current production model's PR-AUC on the latest closed validation window by more than the bootstrap confidence interval — not merely any F1 improvement, which promotes on noise.

COMPONENT 03 · TAKEOVER DETECTORS

Rule-based anomaly flags — Track B

Six transparent detectors: first-time publisher on a version, registry–repo mismatch, a new install hook appearing in a patch release, a surprise young dependency, a dormancy break by a new account, and maintainer handover. Each emits a flag with an explanation string; the combined score is a weighted sum with weights fixed on the design set only.

COMPONENT 04 · SECURITY ASSISTANT

The one real agent

Four tools — get_risk_score, get_takeover_flags, scan_manifest and search_advisories — over retrieval scoped to advisories for the 10,000-package universe rather than all 200,000+ CVE records. A smaller corpus means faster evaluation and the same skill signal. A post-check rejects any answer citing an advisory ID that was not actually retrieved. Open-weights LLM via Groq or Ollama, chosen at build time from what the free tier offers then.

COMPONENT 05 · EVALUATION SERVICE

The differentiator — never cut

Weekly backtest of the risk model on newly closed label windows, RAGAS scoring on a 60-query golden set, GitHub Actions gates that fail the build on faithfulness regression, and Prometheus metrics for both LLM quality and precision decay. This is the component that turns a demo into an LLMops system, and it is the cheapest piece to keep relative to its differentiating value.

Tech stack

COMPONENT	TECHNOLOGY	ROLE	STATUS
Historical backfill	BigQuery public datasets	GH Archive, Scorecard, downloads at scale	NEW
Time series storage	TimescaleDB	Weekly package snapshots, hypertables	NEW

COMPONENT	TECHNOLOGY	ROLE	STATUS
Risk model	LightGBM + SHAP	Ranking and feature attribution	NEW
Experiment tracking	MLflow	Runs, registry, significance-gated promotion	NEW
Vector database	Qdrant	Retrieval over universe advisories	NEW
LLM evaluation	RAGAS	Faithfulness, relevance, context precision	NEW
Container orchestration	Kubernetes (OKE) + Helm	Production deployment	NEW
Pipeline orchestration	Apache Airflow	Ingestion DAGs — late and optional	NEW
LLM backbone	Open weights (Groq / Ollama)	Assistant generation and tool-use	EXISTING
Backend	FastAPI	Async Python API layer	EXISTING
Local development	Docker Compose	Full stack, local dev environment	EXISTING
CI/CD	GitHub Actions	Eval gates, build pipeline, the scanner Action	EXISTING
Monitoring	Prometheus + Grafana	Infra and model quality dashboards	EXISTING

Dropped from version 1, and why

DROPPED	REASON
TFT / darts	This is binary classification on windowed tabular features with few positives, not multi-horizon sequence forecasting. LightGBM trains in minutes and will most likely win. "Why TFT?" had no good answer. Retained only as a stretch goal if it beats LightGBM on the same split.
Prophet baseline	A univariate time-series forecaster is not a baseline for a classification task. Replaced by the three-rung baseline ladder.
NVD as label source	Mapping CPE strings back to PyPI and npm packages is messy and lossy. OSV is ecosystem-native and ships a bulk export per ecosystem.
Live GitHub API for history	Reconstructing metrics at T-90 for thousands of packages would hit rate limits for months. BigQuery does it in queries.
Pre-written metrics	Precision 0.78 and faithfulness 0.89 were invented numbers dressed as results, and implausible at a low base rate. All figures now come from the test set, with confidence intervals, after the build.

The part that decides whether it works

Everything downstream depends on the labeled dataset being honest. This section is the one to build slowly and the one to talk about first in an interview.

Unit of analysis and labels

One row is one package at one weekly snapshot date t . The universe is the top 5,000 PyPI and top 5,000 npm packages by downloads that have a mapped GitHub repository, which keeps crawling tractable and the base rate realistic.

- 1
- For each OSV advisory, compute the **event date E** as the earlier of the advisory's published date and the release date of the first fixed version.
- 2
- A snapshot at t is **positive** if an event date falls in the window $(t + 7 \text{ days}, t + 90 \text{ days}]$.
- 3
- Snapshots with an event in $(t, t + 7 \text{ days}]$ are **dropped**, not labeled negative. This blackout is what stops the model from learning fix activity already underway.
- 4
- All other snapshots are **negative**. Expect a base rate of a few percent — measure it in week 3 and write it down, because it determines which metrics are meaningful.

Leakage controls

- Every feature is computed only from data dated before t , enforced by a unit test that fails if any feature query touches data after it.
- Features that directly encode security work — commits or issues mentioning CVE, security or vulnerability — are excluded, with one deliberate ablation including them to demonstrate the size of the leakage effect.
- Splits are strictly by time: train on label windows closing before 2024-01, validate on 2024-H1, test on 2024-H2 and 2025. Never shuffled at random.
- A package-grouped split is reported alongside, to show the model is not simply memorizing repeat offenders.

Feature families

FAMILY	EXAMPLES — ALL COMPUTED OVER 30/60/180-DAY WINDOWS BEFORE T
Popularity	Downloads, download slope, dependents count, stars
History	Prior advisory count, days since last advisory
Activity	Commit count, active contributors, contributor churn, bus factor
Maintenance	Median issue response time, open-issue ratio, days since last release
Release behaviour	Release count, release-interval variance, major-version jumps
Security practice	Scorecard checks: branch protection, code review, fuzzing, pinned dependencies
Code surface	Language, dependency count, native extensions, package size

The model ladder

Each rung must beat the one below it on the validation window before it earns its place, and every rung is logged to MLflow.

RUNG	MODEL	PURPOSE
B0	Rank by downloads	Popularity baseline — the number to beat
B1	Rank by prior advisory count	History baseline — often surprisingly strong
B2	Logistic regression on popularity + history	Best simple baseline
M1	LightGBM on all feature families	Main model
M2	LightGBM + isotonic calibration	Turns scores into usable probabilities
Stretch	TFT or sequence model	Only if time remains and it beats M1

Metrics and analysis

- **PR-AUC** as the headline metric — ROC-AUC looks flattering at a low base rate.
- **Precision@k and recall@k** for k = 50, 100, 500 per week, matching the real use: which packages should a team review first?
- **Lift over B2** at the same k — the number that answers the thesis.
- **Calibration curve and Brier score** for M2, plus bootstrap confidence intervals on every reported figure.
- **SHAP summary and per-family ablations** showing which signals carry information, plus hand-read error analysis on 20 false positives and 20 false negatives.

Track B evaluation protocol

Design set — used to write and tune the rules: event-stream (npm, 2018), ua-parser-js (npm, 2021), ctx (PyPI, 2022).

Held-out set — touched once, after thresholds are frozen: coa and rc (npm, 2021), colors and faker (npm, 2022), node-ipc (npm, 2022), ultralytics (PyPI, 2024), @solana/web3.js (npm, 2024).

Control set — 500 random top packages over the same period, reported as false alarms per 1,000 package-months. Every incident timeline is rebuilt from primary sources and every date verified before it goes in the README.

Twelve weeks, three phases

Data, then model, then ops, then the assistant, then deployment. Each month ends with a checkpoint that decides what happens next.

Month 1 Leakage-safe dataset and the first honest number

Week 1	Parse the OSV bulk export for PyPI and npm. Map packages to repositories through deps.dev. Fix the universe at the top 10,000 packages by downloads. Done when mapping coverage is measured.
Week 2	BigQuery extraction of GH Archive activity, Scorecard history and downloads into TimescaleDB. Done when one package's full history is queryable end to end.
Week 3	Weekly snapshot table, labels with the seven-day blackout, time-based splits and the leakage unit test. Done when base rate and class counts are recorded.
Week 4	Baselines B0–B2 and the first LightGBM, logged to MLflow. FastAPI shell and Docker Compose stack for every Month 1 service.

Checkpoint 1. If repo-mapping coverage falls below roughly 60% of the universe, shrink to a single ecosystem — npm has the better per-version publisher data. If LightGBM does not beat B2 at all, keep going: the thesis becomes the negative result, and weeks 5 to 7 become the explanation of why.

Month 2 Explanation, takeover detectors and the evaluation loop

Week 5	Feature-family ablations, SHAP attribution, isotonic calibration and hand-read error analysis, written into the repository rather than merely plotted.
Week 6	Build the six takeover detectors against the design set — event-stream, ua-parser-js, ctx — rebuilding each timeline from primary sources.
Week 7	Freeze thresholds, then run the held-out incident test once, with the 500-package control set for false alarms. Results recorded as they come out.
Week 8	Evaluation service: weekly backtest on newly closed windows, the registry promotion gate, Prometheus metrics and the Grafana dashboard.

Checkpoint 2. If you are more than one week behind, apply cuts 1 and 2 before starting the assistant. The evaluation loop ships before the chatbot, always — it is the part that differentiates the project.

Month 3 Assistant, CI gates and deployment

Week 9	Security assistant: four tools, scoped retrieval, the citation post-check, and the orchestrator with routing, retries and a retrieval-only fallback. Done when ten sample queries return valid retrieved citations.
Week 10	A 60-query golden set across factual lookup, risk assessment, comparison and hallucination resistance, with RAGAS wired into GitHub Actions. Done when a regressed prompt fails the build.
Week 11	GitHub Action packaging for requirements.txt and package.json, running on a public demo repository. React dashboard only if time allows.
Week 12	Helm chart and OKE deployment, or Compose if cut. README with real test-set numbers and intervals, architecture diagram, demo video.

Checkpoint 3. No resume bullet goes out without its number coming from the test set, with its confidence interval, written in the README. An unverifiable number is worse than a modest one.

What to cut, and what it is worth when done

Cut in this order if the twelve weeks compress

- 1

Drop the React dashboard — a Grafana dashboard plus the GitHub Action output carries the same signal at a fraction of the cost.
- 2

Drop Airflow — stay on cron and document the DAG design in the README. The orchestration concept is demonstrated either way.
- 3

Drop the Kubernetes migration — stay on Docker Compose with a Helm chart that is written and linted but not deployed. A real loss against the skill gap, but the least damaging to the core narrative.
- 4

Shrink the golden set from 60 queries to 30 before cutting it. A smaller golden set still produces defensible evaluation numbers.
- ×

Never cut the baselines, the leakage test, the time-split evaluation, the registry gate or the CI eval gate. These are what make the project credible; without them it is a student project with a confident README.

Resume bullet templates — fill the brackets from the README after week 12

These are deliberately templates rather than numbers. A modest real result with honest analysis interviews better than a suspiciously clean one, and an interviewer who probes an invented figure ends the conversation there.

- "Built a leakage-controlled dataset of **[N] weekly package snapshots** across [N] PyPI and npm packages from OSV, GH Archive and OpenSSF Scorecard — seven-day blackout window, strict time-based splits, and a unit test that fails on any forward-looking feature query."
- "Trained a LightGBM risk model reaching **PR-AUC [X] versus [Y] for a popularity baseline** ([Z]× lift at top-100), tracked in MLflow with registry promotion gated on bootstrap-significant improvement."
- "Designed six supply-chain takeover detectors — flagged **[N] of [M] held-out npm and PyPI incidents** at [X] false alarms per 1,000 package-months, with thresholds frozen before the held-out test."
- "Shipped a tool-using security assistant with **citation-verified answers**; RAGAS evaluation runs in GitHub Actions and blocks prompt regressions before merge."
- "Packaged a GitHub Action that scans requirements.txt and package.json and returns a ranked risk report inline — a self-demonstrating artifact usable by any engineer."

JD requirement alignment

REQUIREMENT	WHAT AUGUR DELIVERS
RAG + vector databases	Scoped Qdrant retrieval over universe advisories, with a citation post-check
LLM evaluation	RAGAS on a 60-query golden set, gating the CI build
Multi-agent systems	Custom Python orchestrator: routing, shared state, retries, fallback chain
MLOps lifecycle	MLflow tracking, registry, promotion gated on significance rather than noise
Applied ML	LightGBM against three baselines, SHAP attribution, calibration, ablations

REQUIREMENT	WHAT AUGUR DELIVERS
Data engineering	BigQuery backfill, TimescaleDB, Pydantic validation, leakage controls
Production monitoring	Prometheus and Grafana, precision-decay alerts, weekly backtests
Kubernetes	OKE deployment and a Helm chart covering all services

Interview positioning — lead in this order

- 1. The baseline.** "My first question was whether health signals beat popularity at all." That sentence signals scientific thinking before any architecture is described.
- 2. The leakage story.** The blackout window, the time splits and the ablation showing what the metrics look like without them. Very few candidates can discuss leakage from their own project.
- 3. The evaluation loop.** Weekly backtests, the registry gate and the CI gate — how you know it still works next month, not just on demo day.
- 4. Then the architecture.** If XZ Utils comes up, say it inspired the maintainer-trust features but sits outside a PyPI and npm system, and pivot to the event-stream result.